Министерство образования Республики Беларусь
г. Минск
Государственное учреждение образования
«Белорусский государственный университет
информатики и радиоэлектроники»

Лабораторная работа № 5

«Динамическое выделение памяти»
по дисциплине «Основы алгоритмизации и программирования»
Вариант 5

Учебная группа 658304

Выполнил: Дерновский А. П.
Проверил: Селезнев А. И.

Минск 2026

Динамическое выделение памяти

Лабораторная работа № 5

Дерновский А. П.
Основы алгоритмизации и программирования, группа 658304
вариант 5 · Минск, 2026

Цель работы

Научиться динамически выделять память для работы с одномерными и двумерными массивами, в частности с матрицами. Лабораторная работа включает в себя 3 задачи для выполнения. Задачи решаются последовательно.

Во всех трёх задачах память выделяется динамически: количество элементов вводит пользователь, после чего `malloc`/`realloc` выделяют ровно столько памяти, сколько нужно фактическим данным. Массивов переменной длины и глобальных массивов нет. Функции принимают указатели и возвращают `void`; результат работы передаётся через параметры-указатели. Общие операции — чтение чисел и проверка выделения памяти — вынесены в `common.h`: `read_int`, `read_value`, `xmalloc`, `xrealloc`. Заполнение вручную или случайными числами выбирается пользователем там, где это допускает условие.

```c
#ifndef OAIP_COMMON_H
#define OAIP_COMMON_H

#include <stdio.h>
#include <stdlib.h>

static inline void *xmalloc(size_t size)
{
    void *p = malloc(size);

    if (p == NULL && size != 0) {
        fputs("Не хватило памяти.\n", stderr);
        exit(EXIT_FAILURE);
    }
    return p;
}

static inline void *xrealloc(void *p, size_t size)
{
    void *q = realloc(p, size);

    if (q == NULL && size != 0) {
        fputs("Не хватило памяти.\n", stderr);
        exit(EXIT_FAILURE);
    }
    return q;
}

static inline void read_int(const char *prompt, int lo, int hi, int *value)
{
    char line[128], extra;
    int v;

    while (1) {
        fputs(prompt, stdout);
        if (fgets(line, sizeof line, stdin) == NULL) exit(EXIT_FAILURE);
        if (sscanf(line, " %d %c", &v, &extra) == 1 && lo <= v && v <= hi) {
            *value = v;
            return;
        }
        printf("Ошибка: нужно целое число от %d до %d.\n", lo, hi);
    }
```

```c
}

static inline void read_value(const char *prompt, int *value)
{
    char line[128], extra;
    int v;

    while (1) {
        fputs(prompt, stdout);
        if (fgets(line, sizeof line, stdin) == NULL) exit(EXIT_FAILURE);
        if (sscanf(line, " %d %c", &v, &extra) == 1) {
            *value = v;
            return;
        }
        puts("Ошибка: нужно целое число.");
    }
}

#endif

```

Листинг 1. Общий модуль ввода и выделения памяти

Задание № 1

Описание задания. Заполнить массив с клавиатуры. Удалить все элементы, которые меньше предыдущего.

Ход решения. По введённому n выделяется `n` элементов. После ввода массив уплотняется за один проход: первый элемент всегда остаётся, а каждый следующий сравнивается с последним сохранённым. Если он не меньше, он записывается на место `a[k]` и счётчик `k` увеличивается; иначе элемент пропускается. В конце память уменьшается до `k` элементов через `realloc`. В примере 5, 3, 4, 2, 8, 7 остаются 5 и 8: 3, 4 и 2 меньше последнего сохранённого 5, а 7 меньше 8.

```c
#include <stdio.h>

#include "../common.h"

#define MAX_N 100

static void print_array(const char *title, const int a[], int n)
{
    int i;

    printf("%s\n", title);
    for (i = 0; i < n; i++) printf("%5d", a[i]);
    putchar('\n');
}

static void remove_smaller_prev(int **a, int *n)
{
    int i, k = 1;

    for (i = 1; i < *n; i++)
        if ((*a)[i] >= (*a)[k - 1]) (*a)[k++] = (*a)[i];

    *a = xrealloc(*a, (size_t)k * sizeof **a);
    *n = k;
}

int main(void)
{
    int *a, n, i;
    char prompt[32];

    read_int("Размер массива n (1..100): ", 1, MAX_N, &n);
    a = xmalloc((size_t)n * sizeof *a);

    for (i = 0; i < n; i++) {
        snprintf(prompt, sizeof prompt, "a[%d] = ", i);
        read_value(prompt, &a[i]);
    }

```

```
40      print_array("Введённый массив:", a, n);
41      remove_smaller_prev(&a, &n);
42      print_array("Массив после удаления:", a, n);
43      printf("Элементов осталось: %d\n", n);
44
45      free(a);
46      return 0;
47  }
48
```

Листинг 2. Программа к заданию № 1



Рис. 1. Результаты выполнения задания № 1

Задание № 2

Описание задания. В двумерном массиве натуральных чисел продублировать все числа больше K. Количество чисел в строке может быть различным, последнее число — 0.

Ход решения. Матрица хранится как массив указателей на строки `int **a` и отдельный массив длин `int *len`. При ручном вводе строка растёт динамически: вместимость удваивается через `realloc`, а 0 завершает строку и в данные не входит. При случайном заполнении длина строки выбирается от 1 до 8, значения — от 1 до 99. Для дублирования сначала считается число элементов больше K; затем выделяется ровно `len + extra` элементов, строка переписывается с копиями подходящих чисел, а старая освобождается. В примере при $K = 10$ строка 5, 12, 3 становится 5, 12, 12, 3, а строка 20 — 20, 20.



Рис. 2. Результаты выполнения задания № 2

```c
#include <stdio.h>
#include <stdlib.h>

#include "../common.h"

static void fill_ragged(int ***a, int **len, int *n)
{
    int rows, mode, i, j;

    read_int("Количество строк (1..20): ", 1, 20, &rows);
    read_int("Заполнение: 1 - вручную, 2 - случайно: ", 1, 2, &mode);

    *a = xmalloc((size_t)rows * sizeof **a);
    *len = xmalloc((size_t)rows * sizeof **len);
    *n = rows;

    for (i = 0; i < rows; i++) {
        if (mode == 1) {
            int cap = 4, size = 0, v;
            int *row = xmalloc((size_t)cap * sizeof *row);

            for (;;) {
                char prompt[48];
                snprintf(prompt, sizeof prompt, "a[%d][%d] (0 - конец): ", i, size);
                read_value(prompt, &v);
                if (v == 0) break;
                if (v < 0) {
                    puts("Ошибка: нужно натуральное число.");
                    continue;
                }
                if (size == cap) {
                    cap *= 2;
                    row = xrealloc(row, (size_t)cap * sizeof *row);
                }
                row[size++] = v;
            }

            if (size == 0) {
                free(row);
                row = NULL;
            } else {
                row = xrealloc(row, (size_t)size * sizeof *row);
            }
            (*a)[i] = row;
            (*len)[i] = size;
        } else {
```

```c
            int size = rand() % 8 + 1;
            int *row = xmalloc((size_t)size * sizeof *row);

            for (j = 0; j < size; j++) row[j] = rand() % 99 + 1;
            (*a)[i] = row;
            (*len)[i] = size;
        }
    }
}

static void print_ragged(int **a, const int *len, int n)
{
    int i, j;

    for (i = 0; i < n; i++) {
        printf("Строка %d (%d):", i, len[i]);
        for (j = 0; j < len[i]; j++) printf("%4d", a[i][j]);
        putchar('\n');
    }
}

static void duplicate_big(int **a, int *len, int n, int k)
{
    int i, j, extra, new_len, w;

    for (i = 0; i < n; i++) {
        extra = 0;
        for (j = 0; j < len[i]; j++)
            if (a[i][j] > k) extra++;
        if (extra == 0) continue;

        new_len = len[i] + extra;
        int *row = xmalloc((size_t)new_len * sizeof *row);
        w = 0;
        for (j = 0; j < len[i]; j++) {
            row[w++] = a[i][j];
            if (a[i][j] > k) row[w++] = a[i][j];
        }
        free(a[i]);
        a[i] = row;
        len[i] = new_len;
    }
}

static void free_ragged(int **a, int *len, int n)
{
    int i;

    for (i = 0; i < n; i++) free(a[i]);
    free(len);
    free(a);
}

int main(void)
{
    int **a, *len, n, k;

    read_value("K: ", &k);
    fill_ragged(&a, &len, &n);

    puts("Введённый массив:");
    print_ragged(a, len, n);

    duplicate_big(a, len, n, k);
    puts("После дублирования чисел больше K:");
    print_ragged(a, len, n);

    free_ragged(a, len, n);
    return 0;
}

```

Листинг 3. Программа к заданию № 2

Задание № 3

Описание задания. Найти в матрице размером $N \times M$ седловую точку: элемент, минимальный в столбце и максимальный в строке. Удалить столбец, в котором он находится.

Ход решения. Матрица выделяется как N строк по M элементов. Седловая точка ищется построчно: для каждой ячейки проверяется, что в её столбце нет меньшего элемента, а в её строке — большего. При совпадениях значений ячейка тоже считается седловой. Найденный столбец удаляется: для каждой строки выделяется ровно $M-1$ элемент, элементы слева и справа от удаляемого копируются, старая строка освобождается. Если седловых точек нет, матрица остаётся без изменений. В примере 1…9 седловая точка — 3 в правом верхнем углу; после удаления её столбца остаётся матрица 3×2. Если седловых точек нет, программа сообщает об этом, не меняя матрицу.

```c
#include <stdio.h>
#include <stdlib.h>

#include "../common.h"

static void fill_matrix(int ***a, int *n, int *m)
{
    int i, j, mode;

    read_int("Строк n (1..20): ", 1, 20, n);
    read_int("Столбцов m (1..20): ", 1, 20, m);
    read_int("Заполнение: 1 - вручную, 2 - случайно: ", 1, 2, &mode);

    *a = xmalloc((size_t)*n * sizeof **a);
    for (i = 0; i < *n; i++) {
        (*a)[i] = xmalloc((size_t)*m * sizeof ***a);
        for (j = 0; j < *m; j++) {
            if (mode == 1) {
                char prompt[32];
                snprintf(prompt, sizeof prompt, "a[%d][%d] = ", i, j);
                read_value(prompt, &(*a)[i][j]);
            } else {
                (*a)[i][j] = rand() % 101 - 50;
            }
        }
    }
}

static void print_matrix(int **a, int n, int m)
{
    int i, j;

    for (i = 0; i < n; i++) {
        for (j = 0; j < m; j++) printf("%5d", a[i][j]);
        putchar('\n');
    }
}

static int find_saddle(int **a, int n, int m, int *row, int *col)
{
    int i, j, k;

    for (i = 0; i < n; i++) {
        for (j = 0; j < m; j++) {
            int min_col = 1, max_row = 1;
            for (k = 0; k < n; k++)
                if (a[k][j] < a[i][j]) min_col = 0;
            for (k = 0; k < m; k++)
                if (a[i][k] > a[i][j]) max_row = 0;
            if (min_col && max_row) {
                *row = i;
                *col = j;
                return 1;
            }
        }
    }
    return 0;
}

static void delete_col(int ***a, int n, int *m, int col)
{
    int i, j, new_m = *m - 1;

    for (i = 0; i < n; i++) {
        int *row = new_m > 0 ? xmalloc((size_t)new_m * sizeof *row) : NULL;

        for (j = 0; j < col; j++) row[j] = (*a)[i][j];
        for (j = col; j < new_m; j++) row[j] = (*a)[i][j + 1];

        free((*a)[i]);
        (*a)[i] = row;
    }
```

```c
    *m = new_m;
}

static void free_matrix(int **a, int n)
{
    int i;

    for (i = 0; i < n; i++) free(a[i]);
    free(a);
}

int main(void)
{
    int **a, n, m, row, col;

    fill_matrix(&a, &n, &m);
    puts("Введённая матрица:");
    print_matrix(a, n, m);

    if (find_saddle(a, n, m, &row, &col)) {
        printf("Седловая точка: a[%d][%d] = %d\n", row, col, a[row][col]);
        delete_col(&a, n, &m, col);
        printf("Матрица после удаления столбца %d:\n", col);
        if (m == 0)
            puts("Столбцов не осталось.");
        else
            print_matrix(a, n, m);
    } else {
        puts("Седловых точек нет.");
    }

    free_matrix(a, n);
    return 0;
}

```

Листинг 4. Программа к заданию № 3

sehaxe@cachyos: ~/OAiP-Laba5/home

```
sehaxe@cachyos:~/OAiP-Laba5/home$ ./3/main
Строк n (1..20): 3
Столбцов m (1..20): 3
Заполнение: 1 - вручную, 2 - случайно: 1
a[0][0] = 1
a[0][1] = 2
a[0][2] = 3
a[1][0] = 4
a[1][1] = 5
a[1][2] = 6
a[2][0] = 7
a[2][1] = 8
a[2][2] = 9
Введённая матрица:
    1    2    3
    4    5    6
    7    8    9
Седловая точка: a[0][2] = 3
Матрица после удаления столбца 2:
    1    2
    4    5
    7    8
```

Рис. 3. Результаты выполнения задания № 3